Customer_Churn_Analysis_and_Prediction

March 19, 2026

TASK 1

loading the dataset

[2]:
```
import pandas as pd

df = pd.read_csv('/content/sample_data/Telco_Customer_Churn_Dataset (3).csv')
df.head()
```

[2]:
```
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  … DeviceProtection  \
0  No phone service             DSL             No  …               No
1                No             DSL            Yes  …              Yes
2                No             DSL            Yes  …               No
3  No phone service             DSL            Yes  …              Yes
4                No     Fiber optic             No  …               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
```

check the shape and basic info

[3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

The output shows:

7,043 rows (customers) and 21 columns (features)

Most columns are object (text/categorical), a few are numeric (tenure, MonthlyCharges, TotalCharges)

Check for missing values

[4]:
```
df.isnull().sum()
```

[4]:
```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
```

```
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

All columns show 0 missing values, the dataset is clean, no nulls to handle.

TotalCharge is stored as text, let's fix it

[5]:
```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'].isnull().sum()
```

[5]:
```
np.int64(11)
```

As expected, converting TotalCharges revealed 11 hidden missing values (they were blank strings before, not NaN).

Let's fill them in

[6]:
```
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())
```

Now let's encode categorical variables. First, we drop the customerID column since it's not useful for modeling

Reason removing CustomerID

customerID is just a unique identifier for each customer, it's a random label like a name, it has no pattern or meaning that a model can learn from. Including it would only add noise and could cause overfitting

[7]:
```
df.drop(columns=['customerID'], inplace=True)
```

Now let's encode binary categorical columns (Yes/No columns) into 0 and 1

[8]:
```
binary_cols = ['gender', 'Partner', 'Dependents', 'PhoneService',␣
 ↪'PaperlessBilling', 'Churn']
df[binary_cols] = df[binary_cols].apply(lambda x: x.map({'Yes': 1, 'No': 0,␣
 ↪'Female': 0, 'Male': 1}))
```

Now let's handle the remaining categorical columns with more than 2 categories using one-hot encoding

[9]:
```
df = pd.get_dummies(df, columns=['MultipleLines', 'InternetService',␣
↪'OnlineSecurity', 'OnlineBackup',
                                 'DeviceProtection', 'TechSupport',␣
↪'StreamingTV', 'StreamingMovies',
                                 'Contract', 'PaymentMethod'], drop_first=True)
```

Now let's split the dataset into training and testing sets

[10]:
```
from sklearn.model_selection import train_test_split

X = df.drop(columns=['Churn'])
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)
print(X_train.shape, X_test.shape)
```

```
(5634, 30) (1409, 30)
```

The dataset is split into:

Training set: 5,634 rows

Testing set: 1,409 rows

30 features in both

TASK 2

Let's start with the overall churn rate

[11]:
```
churn_rate = df['Churn'].value_counts(normalize=True) * 100
print(churn_rate)
```

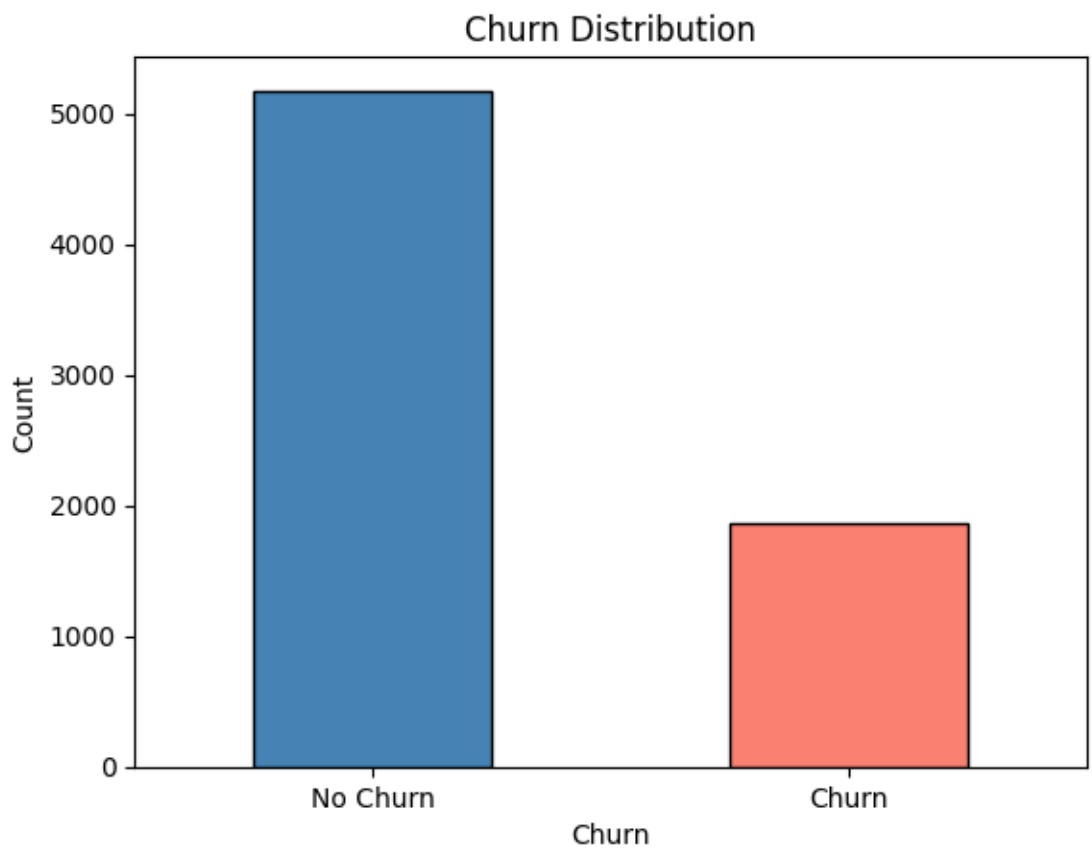
```
Churn
0    73.463013
1    26.536987
Name: proportion, dtype: float64
```

The churn rate is 26.5%, meaning about 1 in 4 customers left the company.

Now let's visualize it:

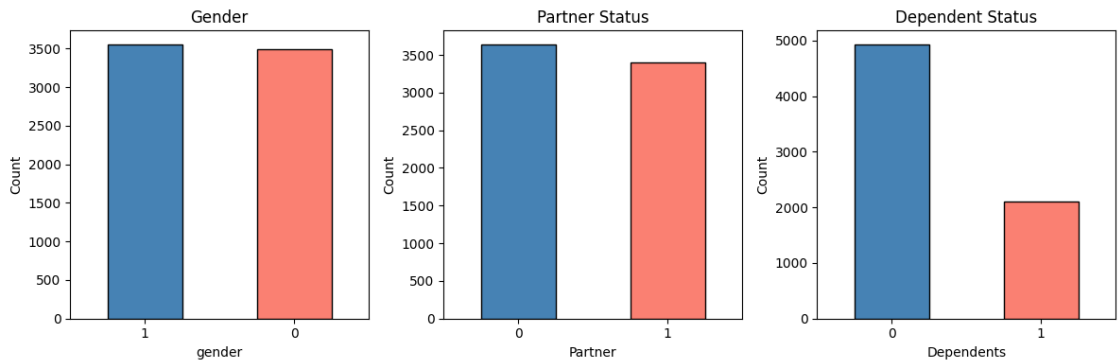
[12]:
```
import matplotlib.pyplot as plt

df['Churn'].value_counts().plot(kind='bar', color=['steelblue', 'salmon'],␣
↪edgecolor='black')
plt.xticks([0, 1], ['No Churn', 'Churn'], rotation=0)
plt.title('Churn Distribution')
plt.ylabel('Count')
```

```
plt.show()
```



We now explore customer distribution by gender, partner, and dependent status

[13]:
```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

for ax, col, title in zip(axes, ['gender', 'Partner', 'Dependents'], ['Gender',␣
↪'Partner Status', 'Dependent Status']):
    df[col].value_counts().plot(kind='bar', ax=ax, color=['steelblue',␣
↪'salmon'], edgecolor='black')
    ax.set_title(title)
    ax.set_ylabel('Count')
    ax.tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()
```



Key observations:

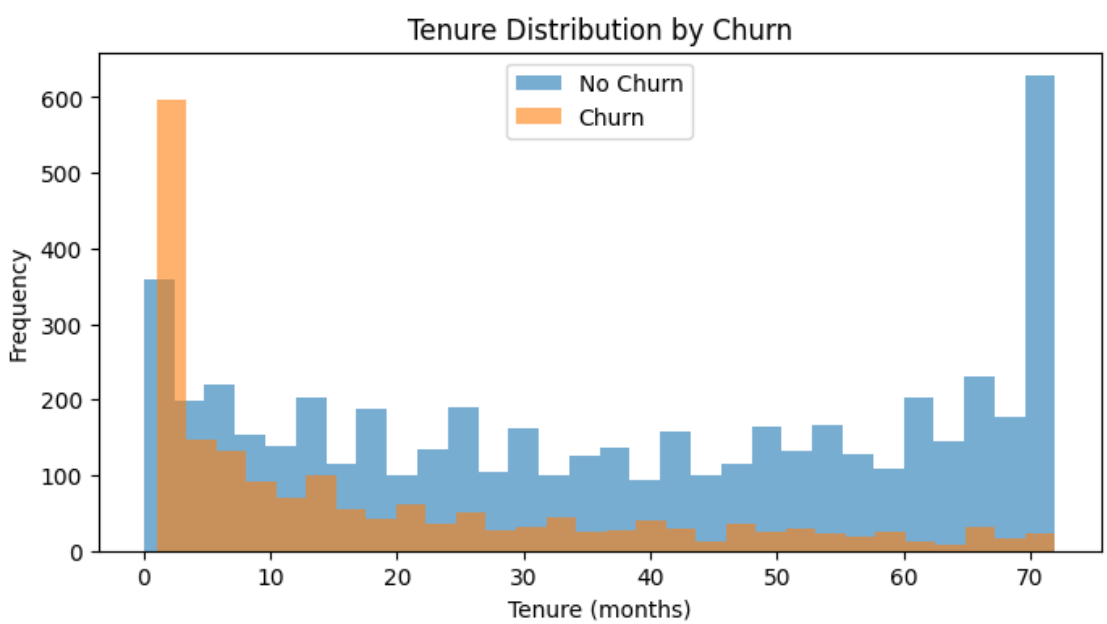
Gender — nearly equal male and female customers

Partner — slightly more customers without a partner

Dependents — majority have no dependents

Now we analyze tenure distribution and its relation with churn

[14]:
```
plt.figure(figsize=(8, 4))
df.groupby('Churn')['tenure'].plot(kind='hist', bins=30, alpha=0.6, legend=True)
plt.title('Tenure Distribution by Churn')
plt.xlabel('Tenure (months)')
plt.legend(['No Churn', 'Churn'])
plt.show()
```

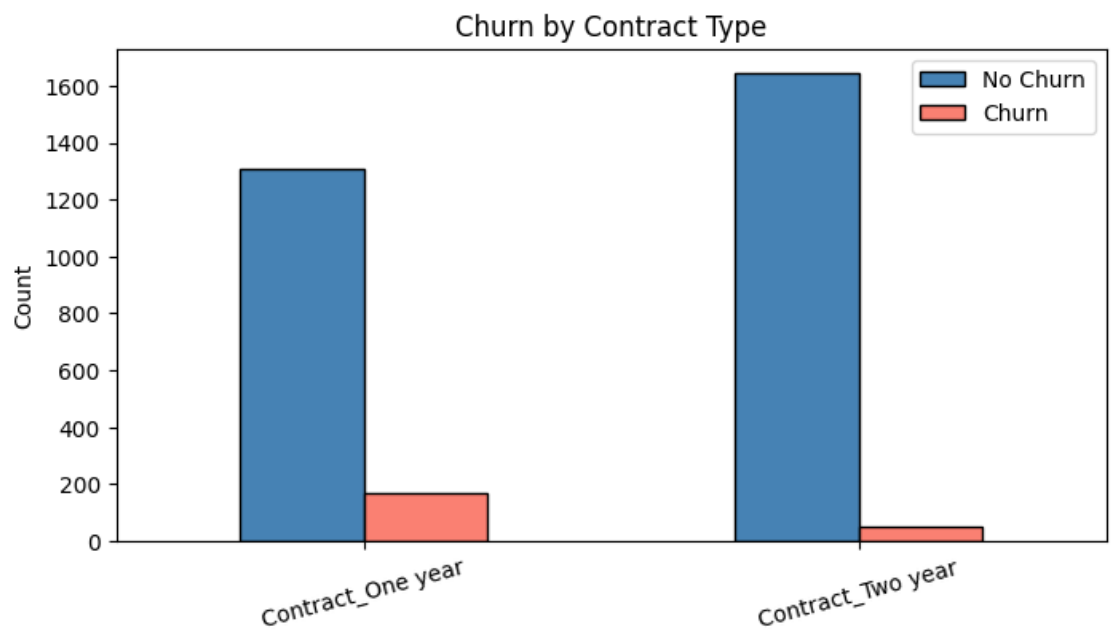


Churned customers are heavily concentrated in the first 1-5 months, new customers are most at risk

Long-tenure customers (60-72 months) rarely churn, loyalty increases over time

Now we investigate churn across contract types

[15]:
```
contract_cols = [col for col in df.columns if 'Contract' in col]
contract_churn = df.groupby('Churn')[contract_cols].sum()
contract_churn.T.plot(kind='bar', figsize=(8, 4), color=['steelblue',␣
↪'salmon'], edgecolor='black')
plt.title('Churn by Contract Type')
plt.ylabel('Count')
plt.xticks(rotation=15)
plt.legend(['No Churn', 'Churn'])
plt.show()
```



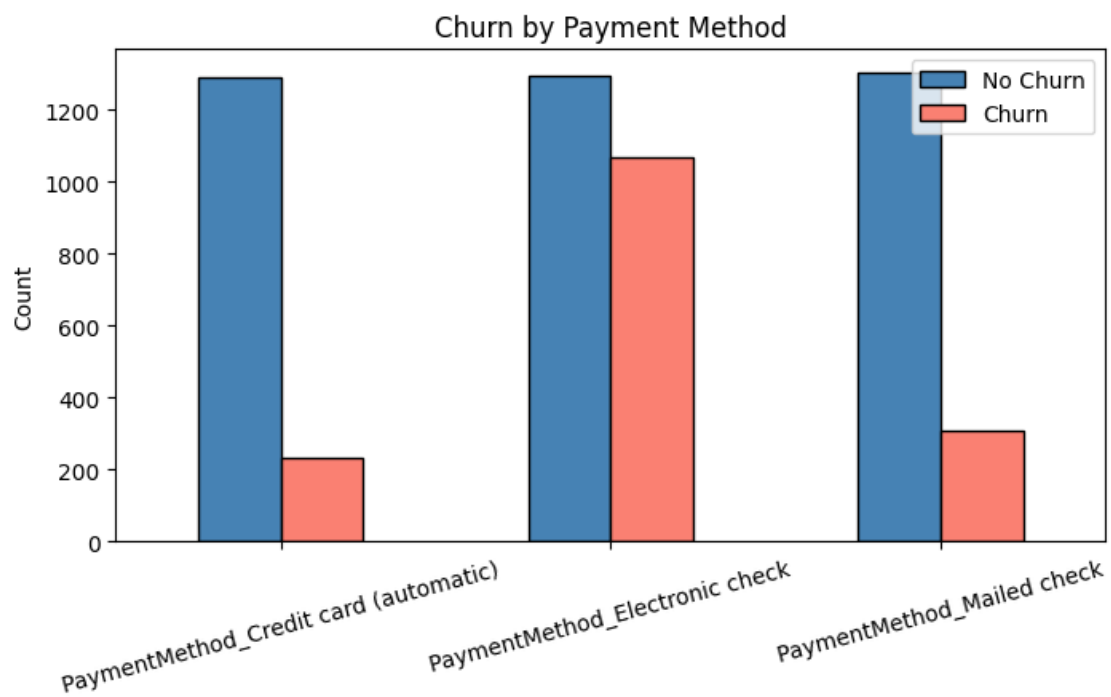
One-year and Two-year contracts have very low churn

The missing bar (Month-to-month, the baseline) is where most churn happens — it was dropped during encoding

investigate churn across payment methods

[16]:
```
payment_cols = [col for col in df.columns if 'PaymentMethod' in col]
payment_churn = df.groupby('Churn')[payment_cols].sum()
```

```
payment_churn.T.plot(kind='bar', figsize=(8, 4), color=['steelblue', 'salmon'],␣
↪edgecolor='black')
plt.title('Churn by Payment Method')
plt.ylabel('Count')
plt.xticks(rotation=15)
plt.legend(['No Churn', 'Churn'])
plt.show()
```



Electronic check has by far the highest churn, nearly as many churned as stayed Credit card (automatic) and Mailed check have much lower churn

The baseline (Bank transfer automatic) also has low churn

TASK 3

We are going to start by segmenting customers based on tenure

[17]:
```
df['tenure_segment'] = pd.cut(df['tenure'], bins=[0, 12, 24, 48, 72],␣
↪labels=['0-12m', '13-24m', '25-48m', '49-72m'])
df['tenure_segment'].value_counts()
```

[17]:
```
tenure_segment
49-72m    2239
0-12m     2175
25-48m    1594
13-24m    1024
```

```
Name: count, dtype: int64
```

we segment by monthly charges

[18]:
```
df['charge_segment'] = pd.cut(df['MonthlyCharges'], bins=[0, 35, 65, 95, 120],␣
↪labels=['Low', 'Medium', 'High', 'Very High'])
df['charge_segment'].value_counts()
```

[18]:
```
charge_segment
High         2604
Low          1735
Medium       1409
Very High    1295
Name: count, dtype: int64
```

Now let's analyze churn rate within tenure segments

[19]:
```
tenure_churn = df.groupby('tenure_segment', observed=False)['Churn'].mean() *␣
↪100
print(tenure_churn)
```

```
tenure_segment
0-12m     47.678161
13-24m    28.710938
25-48m    20.388959
49-72m     9.513176
Name: Churn, dtype: float64
```

churn drops as tenure increases:

0-12 months: 47.7% churn — newest customers are highest risk

13-24 months: 28.7%

25-48 months: 20.4%

49-72 months: only 9.5% — loyal customers rarely leave

We analyze churn rate within charge segments

[20]:
```
charge_churn = df.groupby('charge_segment', observed=False)['Churn'].mean() *␣
↪100
print(charge_churn)
```

```
charge_segment
Low          10.893372
Medium       23.136977
High         35.944700
Very High    32.277992
Name: Churn, dtype: float64
```

higher charges correlate with higher churn:

Low charges: only 10.9% churn

Medium: 23.1%

High: 35.9% — highest risk

Very High: 32.3%

Now let's identify high-value customers at risk of churning:

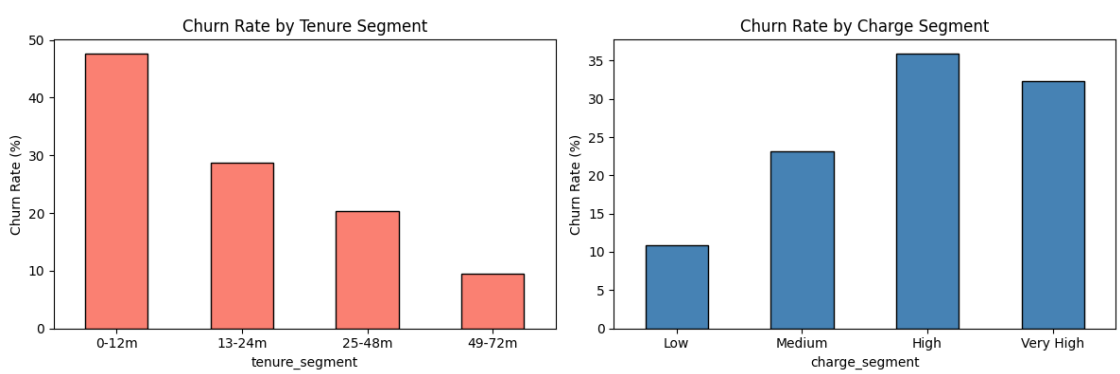
[21]:
```
high_value_at_risk = df[(df['MonthlyCharges'] > 65) &
                        (df['tenure_segment'] == '0-12m') &
                        (df['Churn'] == 1)]
print(f"High-value at-risk customers: {len(high_value_at_risk)}")
```

```
High-value at-risk customers: 664
```

664 high-value customers are at risk — they pay above average but are still in their first year.

Let's visualize churn rates across both segments together:

[22]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

tenure_churn.plot(kind='bar', ax=axes[0], color='salmon', edgecolor='black')
axes[0].set_title('Churn Rate by Tenure Segment')
axes[0].set_ylabel('Churn Rate (%)')
axes[0].tick_params(axis='x', rotation=0)

charge_churn.plot(kind='bar', ax=axes[1], color='steelblue', edgecolor='black')
axes[1].set_title('Churn Rate by Charge Segment')
axes[1].set_ylabel('Churn Rate (%)')
axes[1].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()
```



TASK 4

We Start by scaling the features before training

[23]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Now let's train a Logistic Regression model

[24]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

lr = LogisticRegression(random_state=42)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
print(classification_report(y_test, y_pred_lr))
```

```
              precision    recall  f1-score   support

           0       0.86      0.90      0.88      1036
           1       0.68      0.60      0.64       373

    accuracy                           0.82      1409
   macro avg       0.77      0.75      0.76      1409
weighted avg       0.81      0.82      0.82      1409
```

Good results for Logistic Regression:

Accuracy: 82%

Churn recall: 60% — catches 6 out of 10 churners

Churn F1: 0.64 — room for improvement

we train a Decision Tree mode

[25]:
```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train_scaled, y_train)
y_pred_dt = dt.predict(X_test_scaled)
print(classification_report(y_test, y_pred_dt))
```

```
              precision    recall  f1-score   support

           0       0.80      0.80      0.80      1036
           1       0.45      0.46      0.46       373

    accuracy                           0.71      1409
   macro avg       0.63      0.63      0.63      1409
weighted avg       0.71      0.71      0.71      1409
```

Let's try Random Forest which usually performs better

[26]:
```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       0.82      0.91      0.87      1036
           1       0.65      0.46      0.54       373

    accuracy                           0.79      1409
   macro avg       0.74      0.68      0.70      1409
weighted avg       0.78      0.79      0.78      1409
```

Let's try XGBoost which often performs best on tabular data

[27]:
```
from xgboost import XGBClassifier

xgb = XGBClassifier(random_state=42, eval_metric='logloss')
xgb.fit(X_train_scaled, y_train)
y_pred_xgb = xgb.predict(X_test_scaled)
print(classification_report(y_test, y_pred_xgb))
```

```
              precision    recall  f1-score   support

           0       0.83      0.89      0.86      1036
           1       0.63      0.50      0.56       373

    accuracy                           0.79      1409
   macro avg       0.73      0.70      0.71      1409
weighted avg       0.78      0.79      0.78      1409
```

Let's compare all models side by side:

[28]:
```
from sklearn.metrics import f1_score, accuracy_score

models = {'Logistic Regression': y_pred_lr, 'Decision Tree': y_pred_dt,
          'Random Forest': y_pred_rf, 'XGBoost': y_pred_xgb}

for name, pred in models.items():
    acc = accuracy_score(y_test, pred)
    f1 = f1_score(y_test, pred)
```

```
    print(f"{name}: Accuracy={acc:.2f}, Churn F1={f1:.2f}")
```

```
Logistic Regression: Accuracy=0.82, Churn F1=0.64
Decision Tree: Accuracy=0.71, Churn F1=0.46
Random Forest: Accuracy=0.79, Churn F1=0.54
XGBoost: Accuracy=0.79, Churn F1=0.56
```

Logistic Regression wins with the best accuracy (82%) and churn F1 (0.64)

Now let's tune Logistic Regression with hyperparameter tuning

[29]:
```
from sklearn.model_selection import GridSearchCV

params = {'C': [0.01, 0.1, 1, 10], 'solver': ['liblinear', 'lbfgs']}
grid = GridSearchCV(LogisticRegression(random_state=42), params, cv=5,␣
 ↪scoring='f1')
grid.fit(X_train_scaled, y_train)
print("Best params:", grid.best_params_)
```

```
Best params: {'C': 10, 'solver': 'lbfgs'}
```

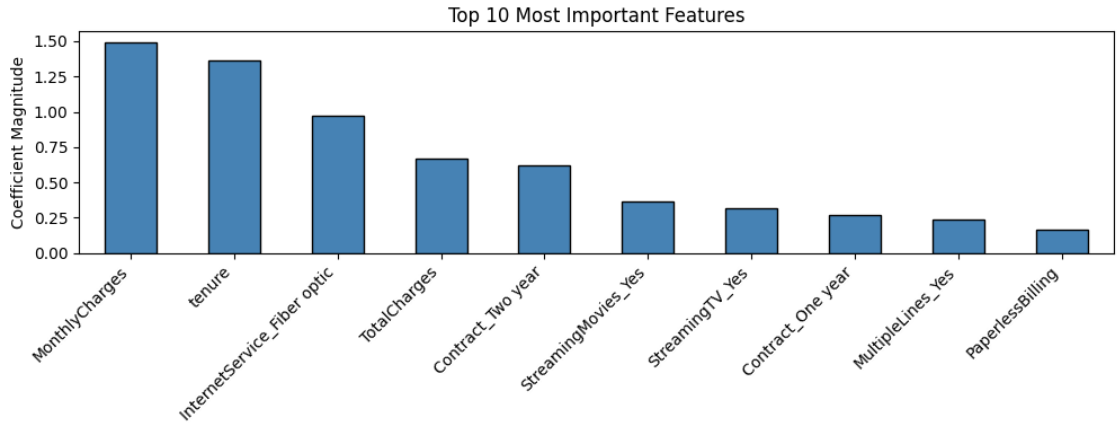
Now let's evaluate the tuned model

[30]:
```
best_lr = grid.best_estimator_
y_pred_best = best_lr.predict(X_test_scaled)
print(classification_report(y_test, y_pred_best))
```

```
              precision    recall  f1-score   support

           0       0.86      0.90      0.88      1036
           1       0.68      0.59      0.63       373

    accuracy                           0.82      1409
   macro avg       0.77      0.75      0.76      1409
weighted avg       0.81      0.82      0.81      1409
```

we do feature selection to find the most important predictors

[31]:
```
import numpy as np

feature_importance = pd.Series(np.abs(best_lr.coef_[0]), index=X_train.columns)
feature_importance.nlargest(10).plot(kind='bar', color='steelblue',␣
 ↪edgecolor='black', figsize=(10, 4))
plt.title('Top 10 Most Important Features')
plt.ylabel('Coefficient Magnitude')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



The top churn predictors are:

MonthlyCharges, highest impact, higher bills drive churn

Tenure, longer customers are less likely to leave

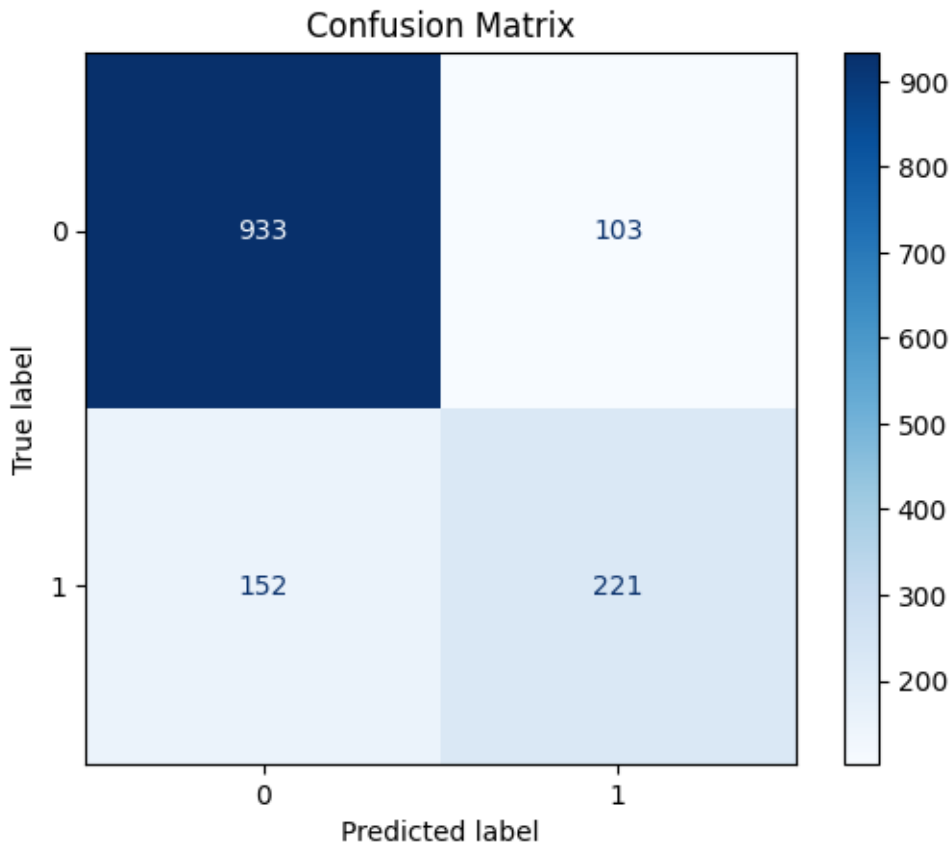
InternetService (Fiber optic), fiber optic customers churn more TotalCharges, overall spend matters

Contract (Two year), long contracts reduce churn

TASK 5

We start my plotting the Confusion matrix

[32]:
```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(best_lr, X_test_scaled, y_test,␣
 ↪cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```



The confusion matrix shows:

933 correctly predicted non-churners

221 correctly predicted churners

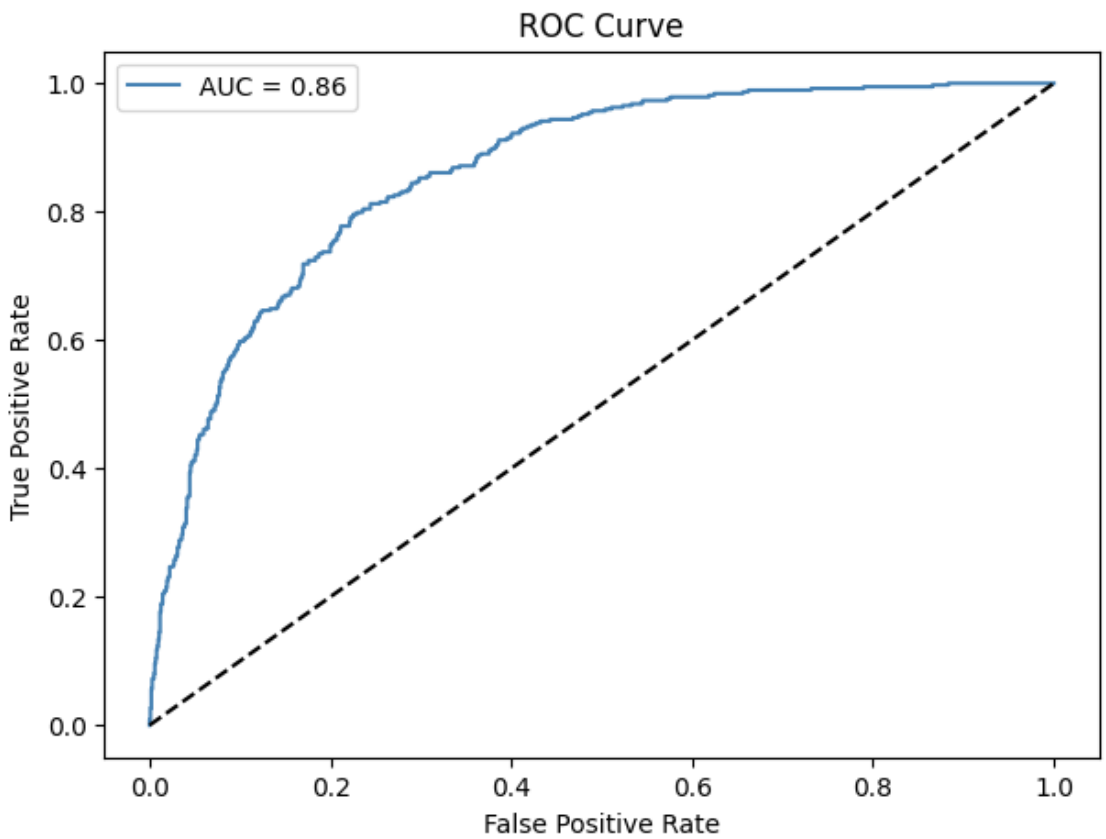
103 false alarms (predicted churn but didn't)

152 missed churners (predicted no churn but did)

we plot the ROC curve and calculate AUC

[33]:
```
from sklearn.metrics import roc_curve, roc_auc_score

y_prob = best_lr.predict_proba(X_test_scaled)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, color='steelblue', label=f'AUC = {auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.title('ROC Curve')
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```

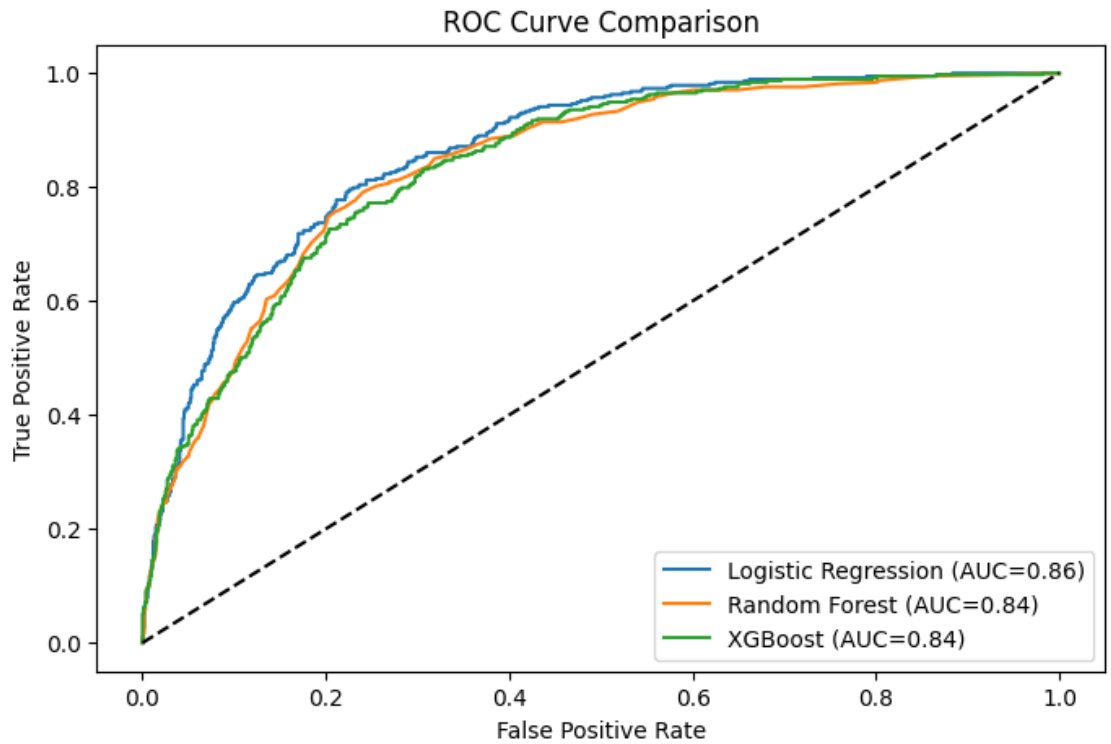


AUC = 0.86 is a strong result meaning the model has an 86% chance of correctly distinguishing a churner from a non-churner

We compare ROC curves for all models

[34]:
```
plt.figure(figsize=(8, 5))

for name, model in [('Logistic Regression', best_lr), ('Random Forest', rf),␣
 ↪('XGBoost', xgb)]:
    y_prob = model.predict_proba(X_test_scaled)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc:.2f})')

plt.plot([0, 1], [0, 1], 'k--')
plt.title('ROC Curve Comparison')
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```



Logistic Regression confirmed as the best model:

Logistic Regression: AUC = 0.86

Random Forest: AUC = 0.84

XGBoost: AUC = 0.84

TASK 6

Lets calculate the revenue at risk from churners first

[35]:
```
churners = df[df['Churn'] == 1]
revenue_at_risk = churners['MonthlyCharges'].sum()
print(f"Monthly revenue at risk: ${revenue_at_risk:,.2f}")
```

```
Monthly revenue at risk: $139,130.85
```

Now let's estimate revenue saved if we retain 30% of at-risk customers:

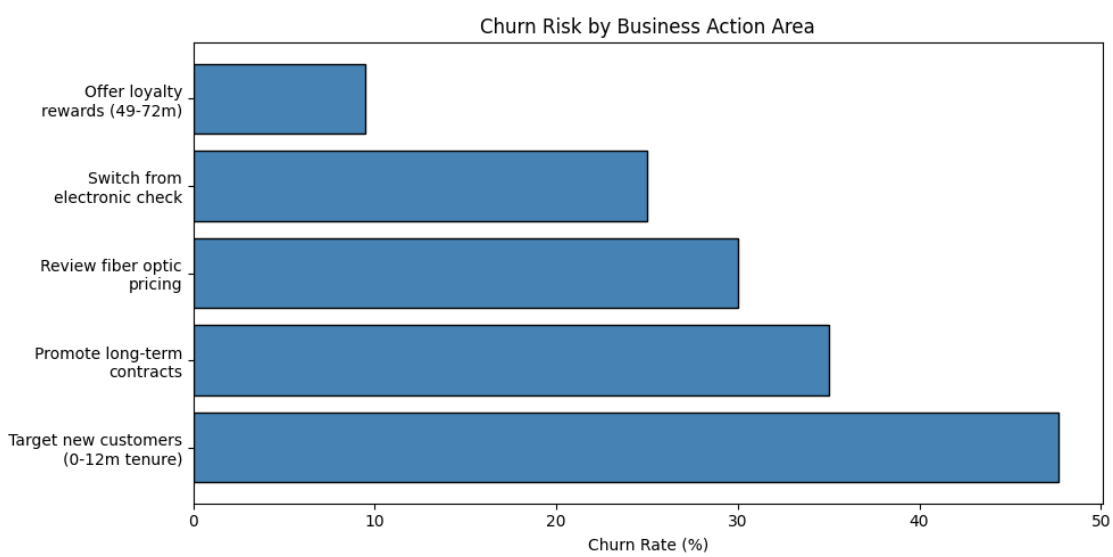
[36]:
```
retention_rate = 0.30
saved_revenue = revenue_at_risk * retention_rate
print(f"Potential monthly revenue saved: ${saved_revenue:,.2f}")
print(f"Potential annual revenue saved: ${saved_revenue * 12:,.2f}")
```

```
Potential monthly revenue saved: $41,739.25
Potential annual revenue saved: $500,871.06
```

Retaining just 30% of churners could save $500,871 annually

Now let's visualize the business recommendations

[37]:
```
recommendations = {
    'Target new customers\n(0-12m tenure)': 47.7,
    'Promote long-term\ncontracts': 35.0,
    'Review fiber optic\npricing': 30.0,
    'Switch from\nelectronic check': 25.0,
    'Offer loyalty\nrewards (49-72m)': 9.5
}

plt.figure(figsize=(10, 5))
plt.barh(list(recommendations.keys()), list(recommendations.values()),␣
 ↪color='steelblue', edgecolor='black')
plt.title('Churn Risk by Business Action Area')
plt.xlabel('Churn Rate (%)')
plt.tight_layout()
plt.show()
```



1 Data-Driven Business Recommendations

1.1 Revenue at Risk

- Monthly revenue at risk: $139,130.85
- Potential monthly savings: (30% retention),41,739.25
- Potential annual savings: (30% retention), $500,871.06

1.2 Recommendations

1.2.1 1. Target New Customers (0–12 Months)

- Churn rate: **47.7%** — highest risk group
- Assign dedicated onboarding support in the first 90 days
- Offer welcome discounts and check-in calls to build early loyalty

1.2.2 2. Promote Long-Term Contracts

- Month-to-month customers churn the most
- Offer discounts or incentives to switch to 1-year or 2-year contracts
- Long-term contract customers churn at a fraction of the rate

1.2.3 3. Review Fiber Optic Pricing

- Fiber optic is the **3rd biggest churn predictor**
- Investigate whether pricing or service quality is driving dissatisfaction
- Consider competitive pricing adjustments or service upgrades

1.2.4 4. Discourage Electronic Check Payments

- Electronic check users have the **highest churn** among all payment methods
- Incentivize auto-pay via credit card or bank transfer
- Offer a small monthly discount for switching to automatic payment

1.2.5 5. Reward Loyal Customers (49–72 Months)

- Churn rate drops to just **9.5%** at this stage
- Maintain loyalty programs to keep long-tenure customers engaged
- Prevent late-stage exits with exclusive perks and renewal offers

---

1.3 Projected Impact

Action	Target Group	Expected Churn Reduction
Onboarding program	0–12 month customers	~10–15%
Long-term contract incentives	Month-to-month customers	~8–12%
Fiber optic pricing review	High-charge customers	~5–8%

Action	Target Group	Expected Churn Reduction
Auto-pay incentives	Electronic check users	~5–7%
Loyalty rewards	49–72 month customers	~2–3%